IronLot — Task 1 Report

Automated Bidding Model for Used Car Auctions

Harsh Srivastava | IIT Guwahati | B.Tech Mechanical Engineering

---

Overview

So the task basically had two parts — first train a model that can predict what a used car will sell for at auction just from its features, and second build a bot that actually goes into a live simulation and bids on cars with a $500,000 budget, trying to buy below predicted value and make profit against rival agents.

I went with LightGBM as the main model and used Optuna for hyperparameter tuning. Final validation RMSE came out to $2,042 with R-squared of 0.955, which means the model explains about 95.5% of why one car sells for more than another.

---

1. Data Cleaning

First thing I did was look at what's actually wrong with the data. There were missing values across almost every column, and some entries that were clearly wrong.

Missing values:

Column	Missing	%	What I did and why
transmission	52,299	11.7%	Filled with Unknown — has its own price pattern, mode would be wrong
color	20,361	4.6%	Filled with Unknown
interior	14,155	3.2%	Filled with Unknown
body	10,593	2.4%	Filled with Unknown
condition	9,437	2.1%	Filled with median — has outliers so mean would be skewed
make / model	~8,300	1.9%	Filled with Unknown
odometer	69	0.01%	Filled with median

Why Unknown and not mode for categoricals?

For transmission especially, I checked the median price for automatic vs manual vs Unknown separately and they were all different. So Unknown cars have their own price behavior — they tend to be older or lower quality vehicles. If I filled with mode (automatic), the model would incorrectly learn the pricing patterns of automatic cars for these entries. Keeping Unknown as its own category gives the model better information.

Why median and not mean for numerics?

Odometer had values like 999,999 miles in it — that's a sensor reset on old cars, not a real reading. These extreme values pull the mean way up. The median is not affected by outliers so it gives a much more realistic imputation value.

Outlier removal:

Selling price: Found 4 cars priced at $1, $2, $4 — obviously data entry errors. On the upper end, 127 cars above $100k are ultra luxury vehicles like Rolls-Royce and Ferrari. They're less than 0.03% of the

data but create a massive price range that hurts RMSE for the 99.97% of normal cars. So I filtered to $500 - $150,000.

Odometer: Capped at 300,000 miles. Anything above that is a sensor glitch — the data had entries showing 999,999 which is physically not realistic.

---

2. EDA — What I Found

I tried to go beyond basic bar charts and find patterns that would actually affect bidding decisions. Here are the four most interesting things I found.

Depreciation is different across body types

When I plotted price against car age separately for each body type I noticed SUVs and Pickups hold value significantly better than Sedans after about age 3-4. By age 8 the gap between a Pickup and a Sedan of the same age is nearly $8,000. This means body type and age interact — they're not independent features.

Condition has a non-linear effect on price

I expected a smooth linear relationship but the data showed a threshold effect — price drops very steeply for condition below 2.0, but the difference between condition 3.0 and 4.0 is much smaller than between 1.5 and 2.0. This is one reason I went with a tree based model over linear regression — trees can capture these thresholds naturally.

Odometer per year is more useful than raw odometer

Two cars can both have 80,000 miles but one is 3 years old (driven really hard) and the other is 10 years old (normal usage). When I grouped cars into usage quintiles by odo_per_year, the price gap between Very Low and Very High usage was over $6,000 — larger than what raw odometer alone explained for newer cars.

Transmission x body type combinations beat segment average

Manual Pickups sell around 8-12% above the Pickup segment average price. This shows that transmission is not just a standalone feature — its effect on price depends on body type. This motivated the interaction features I added.

---

3. Feature Engineering

Based on the EDA I created 4 new features on top of the original columns. All 8 text columns were label encoded and the encoders were saved separately so the agent can use them for single-row predictions.

Feature	Formula	Why I made it
car_age	2015 - year	Depreciation depends on age not absolute year. Clipped at 0 for some future yea
odo_per_year	odometer / car_age	Usage intensity. Much better signal than raw odometer especially for newer cars.
condition_x_age	condition x car_age	An old car in bad condition is way worse than just old or just bad separately. Cap
low_odo_good_cond	(odo < 30k) x condition	Captures the nearly-new premium. Only activates when BOTH mileage is low AN

---

4. Model Training and Hyperparameter Tuning

Why LightGBM?

Main reason I went with LightGBM was the data had 954 unique car models and 93 unique makes — really high cardinality. Linear models would need one-hot encoding for all of these which explodes the feature space and becomes slow. LightGBM handles high cardinality categoricals natively through label encoding and tree splits.

I also tried a baseline Ridge Regression first just to have a comparison point — that gave RMSE around $4,823. LightGBM with default params brought it down to $2,155. After tuning, final RMSE is $2,042.

Why Optuna over GridSearchCV?

With 8 hyperparameters each having a continuous range, grid search would need millions of combinations — would take days. Optuna is smarter, it learns from each trial and searches in more promising directions. 30 trials was enough to converge to good params in about an hour of runtime.

Parameters tuned and results:

Parameter	Range	Best Value	What it controls
n_estimators	400-1200	800	Number of trees in the ensemble
learning_rate	0.02-0.10	0.05	How much each tree corrects the previous one
num_leaves	63-255	191	Complexity of each individual tree
min_child_samples	10-80	20	Main overfitting control — min data per leaf
subsample	0.6-1.0	0.80	Fraction of rows used per tree — adds randomness
colsample_bytree	0.5-1.0	0.80	Fraction of features used per tree
reg_alpha (L1)	0.0001-10	0.10	Pushes weak feature weights to zero
reg_lambda (L2)	0.0001-10	0.10	Shrinks all weights slightly — smooth regularization

Model comparison:

Model	Val RMSE	R-squared	Notes
Ridge Regression	$4,823	0.831	Baseline — simple linear model
LightGBM default params	$2,155	0.950	Already much better out of the box
LightGBM tuned (Optuna 30 trials)	$2,042	0.955	Final submitted model

5. Bid Sequence Design

The bidding logic is a pure deterministic function of 4 inputs — predicted value (V), bankroll (B), current highest bid (H), and round number (R). Same inputs always give the same bid, no randomness.

The formula:

Discount based on round — how much below fair value we'll pay:

```
discount = 0.93 if R <= 3 | 0.90 if R <= 6 | 0.87 if R > 6
```

```
max_bid = min( V x discount, B x 0.20 )
```

If current bid already above our max — pass:

```
if H >= max_bid: return 0
```

Otherwise calculate increment and bid:

```
aggression = 0.50 if R <= 3 | 0.35 if R <= 6 | 0.20 if R > 6
increment = max(100, (max_bid - H) x aggression)
bid = round_to_nearest_50( H + increment )
```

Why discount and aggression change with round number:

Early rounds (1-3) — more rivals are still in the auction so I set higher discount (0.93) and higher aggression (0.50 of gap). This closes deals faster instead of dragging out long bidding wars where rivals can chip away at your budget.

Late rounds (7+) — most rivals have either won their car or dropped out. I tighten to 0.87 discount and only jump 20% of the remaining gap. Small increments here bait remaining rivals into going past their own ceiling before we do.

Why this beats flat increments:

A flat $500 increment has two problems — early on it's too slow and rivals close the deal before we react, and late in the auction $500 can be unnecessarily large when rivals are close to their max. The fractional approach adjusts the increment size based on how much room is actually left, so it's always the right size for the situation.

Aggression vs margin tradeoff:

Discount	Aggression	Result
0.95+ (pay 95% of value)	High (0.60)	More wins but very thin margins — risky if predictions are off
0.87 (pay 87% of value)	Low (0.20)	Fewer wins but strong profit on each win
0.87-0.93 blended (my approach)	0.20-0.50 mixed	Balanced — targets roughly 10% margin while staying competitive

Budget protection:

The 20% bankroll cap per car is really important. Even if the model is completely wrong on one car — say predicts $20,000 but actual value is $8,000 — the most we spend is $4,000 (20% of a $20,000 bankroll scenario). We still have 80% left to keep participating. Without this cap one bad prediction could effectively end our auction.

---

6. Submitted Files

File	What it does
agent_Harsh.py	The actual auction bot — loads model + encoders, implements analyze_item() and place_bid()
model_Harsh.pkl	Trained LightGBM model with feature list and current_year saved together
encoders_Harsh.pkl	All 8 LabelEncoders fitted on training data — needed for single-row inference in the arena
analysis_Harsh.ipynb	Full EDA, feature engineering, model training and evaluation notebook
report_Harsh.pdf	This document

All pkl files are loaded using relative paths in agent_Harsh.py to ensure compatibility with the Arena simulation environment.